

Implementação baseada em Redis — relatório técnico

Projeto: sistemas-distribuidos-2025.2

5 de dezembro de 2025

Resumo

Este documento descreve a estratégia, arquitetura, metodologia de testes (protótipos), infraestrutura e cenários no modelo SBC para a implementação baseada em Redis usada no projeto localizado na pasta **backend/** da workspace. O documento está escrito em português e foi adaptado ao código do repositório (uso de Redis para gerenciamento de sessões).

1 Estratégia

Objetivo: utilizar Redis como uma camada leve de armazenamento de estado (sessões) e cache para permitir sessões escaláveis e rápidas sem persistir estado sensível no servidor principal.

Principais decisões:

- Usar Redis para sessões (via **connect-redis**) — sessão centralizada, compartilhada entre instâncias do backend.
- Executar Redis em container (Docker) para facilitar reprodução do ambiente.
- Proteger acesso ao Redis quando o porto do container é publicado no host (usar autenticação com **--requirepass** e publicar apenas em **127.0.0.1** durante desenvolvimento).
- Configurar o backend para ler a URL do Redis via variável de ambiente **REDIS_URL** (*padrão* : **redis://:PASSWORD@127.0.0.1:6379**).

2 Arquitetura

A arquitetura adotada é simples e eficaz para o propósito do projeto: Frontend (browser) → Backend (Express) → Redis (sessões). Abaixo está um diagrama simplificado.

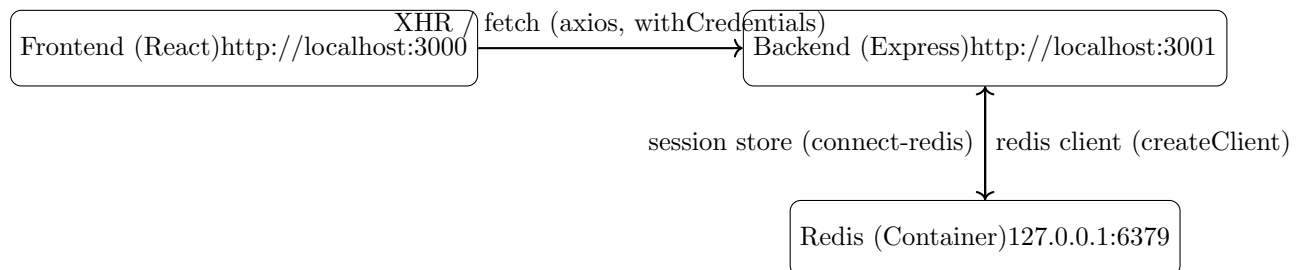


Figura 1: Visão arquitetural simplificada

Observações específicas do projeto:

- O backend atual define a URL do redis em **server.js** (exemplo: **redis://:your-strong-dev-pass@localhost**). Verifique a variável de ambiente **REDIS_URL** em *produção/testes.Assessões* *salvas* *com prefixo sess: por padrão* *redis*). Use comandos **SCAN/GET** no redis — clique para inspecionar.

3 Infraestrutura

Recomenda-se executar Redis via Docker Compose durante o desenvolvimento. Exemplo mínimo (adaptado do repositório):

```
# backend/docker-compose.yml (trecho)
version: '3.8'
services:
  redis:
    image: redis:7-alpine
    command:
      - redis-server
      - --bind
      - 0.0.0.0
      - --requirepass
      - "<your-dev-pass>"
      - --protected-mode
      - "yes"
    ports:
      - "127.0.0.1:6379:6379" # bind to host loopback only for
                             development
```

Notas de segurança:

- Em desenvolvimento, publicar em 127.0.0.1 impede acesso direto da rede externa.
- Em produção, evite publicar Redis ao público; prefira networking interno (sem **ports:**) e autenticação forte.
- Habilite ACLs/TLS em ambientes expostos.

4 Metodologia de testes (protótipos)

Objetivo: validar comportamento de sessões, latência e tolerância a falhas para o uso de Redis. A estratégia de testes segue três níveis:

1. Testes unitários

- Testes do código do backend (rotas, validação) com mocks do Redis. Use bibliotecas de testes do Node.js (Jest, Mocha). - Mock do client Redis para verificar que a função de criação de sessão chama o método `req.session.save` corretamente.

2. Testes de integração (local)

- Subir Redis via docker-compose e executar testes que fazem requests reais ao backend. - Verificar que, após login, existe uma chave `sess:*` no Redis. Comandos úteis:

```
# check keypace
redis-cli -h 127.0.0.1 -p 6379 -a '<your-dev-pass>' INFO keypace
# scan sessions
redis-cli -h 127.0.0.1 -p 6379 -a '<your-dev-pass>' --scan --pattern
'sess:*
```

3. Testes de carga / protótipo

Cenários:

- 1) Conexões concorrentes de login: medir latência e throughput quando N usuários tentam logar simultaneamente (por exemplo, N=100, 500, 1000).
- 2) Persistência de sessão: conectar, confirmar sessão, reiniciar backend e confirmar que a sessão persiste em Redis (se esperado).
- 3) Falha e recuperação do Redis: simular reinício do container Redis e verificar comportamento do backend (reconexão, erros de sessão).

Ferramentas de teste:

- Artillery (node) ou hey / wrk para carga HTTP.
- redis-benchmark para testes específicos de throughput em Redis.

Exemplo simples de comando Artillery (fluxo de login):

```
# Exemplo (instale artillery globalmente: npm i -g artillery)
# arquivo simple-login.yml
config:
  target: "http://localhost:3001"
  phases:
    - duration: 60
      arrivalRate: 20
  scenarios:
    - flow:
      - post:
          url: "/api/auth/login"
          json:
            username: "user"
            password: "user123"
```

5 Cenários no modelo SBC (interpretação)

Nota: aqui adotamos uma interpretação prática de "SBC" para este documento: "Servidor (Backend) — Buffer (Redis) — Cliente (Frontend)". Ou seja, SBC descreve o fluxo entre cliente, buffer de sessão e servidor que processa lógica e autenticação.

Cenário 1: Login simples

1. Cliente envia POST /api/auth/login.
2. Backend valida credenciais consultando `data/users.json` (arquivo local no projeto).
3. Backend cria sessão em Redis (chave `sess:<id>`) usando `req.session` e responde com Set-Cookie.
4. Cliente armazena cookie; em requests subsequentes envia cookie e backend autentica via sessão lida do Redis.

Cenário 2: Multicliente concorrente

- Objetivo: observar impacto no tempo de resposta e latência por sessão quando 100/500 clientes simultâneos fazem login.
- Métricas: throughput, latência 50/95/99 perc., taxas de erro, número de chaves no Redis.

Cenário 3: Reinício do backend (sem perder sessão)

1. Com sessão criada no Redis, reinicie o processo do backend.
2. Verificar que clientes com cookie válido continuam autenticados (backend lê a sessão do Redis).

Cenário 4: Falha temporária do Redis

1. Simule parada do Redis (docker stop) e observe as reações do backend (logs, erros de conexão).
2. Verificar que operações críticas falham com mensagens de erro apropriadas e que, após o reinício do Redis, o backend reconecta e restaura funcionalidade.

6 Checklist de implantação / comandos úteis

1. Subir Redis no host (bind em loopback e com senha para dev):

```
sudo docker-compose -f backend/docker-compose.yml up -d
```

2. Rodar backend no host (usa `REDIS_URL`):

Conclusão e próximos passos

- Esta proposta oferece um roteiro para usar Redis como buffer de sessões no projeto com segurança local (bind em 127.0.0.1) e testes práticos para validar comportamento sob carga.
- Próximo passo opcional: eu posso gerar um arquivo de configuração Artillery para os cenários de carga e patchar `backend/server.js` para usar `process.env.REDIS_URL` em produção.